Инструмент исследования бинарного протокола поверх TCP

Настольное приложение для разбора недокументированного бинарного протокола поверх TCP. Оно открывает захват трафика формата PCAP или PCAPNG, выделяет TCP-сессии, собирает направленные потоки, показывает байты с привязкой к пакетам, позволяет выдвигать гипотезы о структуре сообщений, применять декларативные правила интерпретации, проверять их на корпусе, находить контрпримеры и сохранять всё исследование переносным проектом с отчётом.

Главный принцип: факт и догадка не смешиваются. Каждый диапазон байт в восстановленном потоке связан с породившим его пакетом; пропуск или неоднозначное наложение показываются как есть, а не заполняются; правило, согласованное с примерами, считается подтверждённым в проверенном объёме, а не доказанным.

Интерфейс доступен на русском и английском языке (переключается в меню языка, выбор запоминается) и в тёмной и светлой теме (меню «Вид», тёмная по умолчанию). Все команды работают офлайн.

Реализованная функциональность

- **Чтение захватов.** PCAP и PCAPNG, включая несколько блоков интерфейса, разные типы канального уровня и разбор временных меток.
- **TCP-сессии.** Идентификация соединений по четвёркам адрес/порт, отслеживание открытия, закрытия (FIN) и сброса (RST), порядок потоков.
- **Потоковая сборка.** Восстановление каждого направления по номерам последовательности, диагностика пропусков и неоднозначных перекрытий, отметка повторных передач.
- **Привязка к пакетам.** Для каждого байта восстановленного потока хранится ссылка на исходный пакет (номер, файл, смещение, временная метка).
- **Нормализованный экспорт.** Единый JSON-контракт между всеми стадиями (`docs/CONTRACT.md`), пригодный для внешних инструментов.
- **Потоковый режим.** Нормализация больших захватов без удержания всего файла в памяти.
- **Экспорт в Wireshark.** Запись восстановленных потоков обратно в pcap.
- **Фрейминг сообщений.** Несколько стратегий выделения границ сообщений (по префиксу длины, разделителю, фиксированному размеру, длине с конца).
- **Декларативные правила.** Описание структуры сообщения в версионируемом формате; движок применяет правило ко всему потоку или корпусу и выдаёт машинно-читаемый результат.
- **Типы полей.** Целые со знаком и без, числа с плавающей точкой (`float`), переменная длина (`varint`), структуры «тег — длина — значение» (`tlv`) и битовые поля.
- **Машинно-читаемые выгрузки.** Результат применения правила пишется как JSON-контракт, как построчный JSONL или как CSV по колонкам полей.
- **Пары запрос и ответ.** Сообщения обоих направлений связываются в ходы с указанием уверенности и перечислением наблюдений, на которых она основана.
- **Заметки исследователя.** Пометки на диапазоне байтов хранятся в проекте (SQLite) и попадают в отчёт.

- **Проверка и контрпримеры.** Сверка правила с корпусом, классификация сообщений (`matched`, `mismatched`, `incomplete`, `uncovered`), показ конкретных байт, опровергающих гипотезу.
- **Версионирование.** Правила и результаты версионируются; поддерживается сравнение двух версий правила и двух результатов.
- **Альтернативные гипотезы.** Панель гипотез оценивает конкурирующие прочтения полей по всему корпусу, а не по отдельному примеру.
- **Переносной проект.** Исследование сохраняется на диск каталогом с манифестом (относительные пути и sha256), экспортируется и импортируется zip-архивом с проверкой целости.
- **Отчёты.** Исследование отрисовывается в Markdown и в самодостаточную HTML-страницу.
- **Графический интерфейс.** Дерево сессий, шестнадцатеричный вид с привязкой к пакетам, сравнение сообщений, панель правила и валидации, панель гипотез, сравнение версий, диалог настроек.
- **Интерфейс и темы.** Русский и английский язык, тёмная и светлая тема, размеры шрифтов, каталоги по умолчанию и уровень логирования — в диалоге настроек.
- **Веб-интерфейс (необязательно).** Локальная страница поверх тех же движков: сессии, шестнадцатеричный вид с вердиктами, применение правила, провенанс байта, экспорт HTML-отчёта. Только `127.0.0.1`, состояние в памяти.

Особенность проекта в следующем

1. **Байт всегда связан с пакетом.** Привязка провенанса не надстройка, а свойство самой модели потока: любой байт можно раскрыть до пакета, из которого он пришёл. Это делает вывод проверяемым, а не правдоподобным.
2. **Совпадение не считается доказательством.** Правило, согласованное со всеми примерами, отчитывается как подтверждённое в проверенном объёме. Корпус целенаправленно содержит контрпример — команду, которую узкое правило v1 не допускало и которая уточнила правило до v2.
3. **Пропуск виден, а не заполнен.** Пропуски и неоднозначные перекрытия в потоке не маскируются: они показываются отдельными категориями с координатами и остаются входными данными для следующего шага.

Основной стек технологий

Слой	Технологии
Язык	Python 3.12+
Интерфейс	PySide6 6.8.1 (Qt 6), свой набор токенов темы, шрифты Tektur и Montserrat
Разбор захватов	dpkt 1.9.8
Конфигурация правил	JSON, JSON Schema (`jsonschema`)
Тесты	pytest, pytest-cov
Качество кода	ruff
Сборка	setuptools, PyInstaller, dpkg-deb, Docker

Графический интерфейс, разбор захватов, движок правил и отчёты не зависят от сторонних сетевых сервисов: приложение полностью локальное.

Обоснование технологического стека

Каждый выбор ниже сделан под эталонную нагрузку задания (100 МиБ, 250k пакетов, 1000 сессий, 8 ГБ ОЗУ) и под требование воспроизводимости на другой машине.

- **Разбор захватов — `dpkt`, а не PyShark.** PyShark запускает `tshark` и разбирает его текстовый вывод: это внешняя зависимость, лишний процесс на каждый файл и потеря деталей, которые нужны для точного восстановления потока. `dpkt` — чистая библиотека на Python, читает PCAP и PCAPNG напрямую, даёт доступ к sequence numbers, флагам и сырым байтам без промежуточного слоя. Это делает разбор детерминированным, пригодным для Docker-образа без `tshark` и воспроизводимым на машине без Wireshark.
- **Движок правил — собственный, а не pandas.** Разбор идёт потоком: файл читается пакет за пакетом, поток собирается по sequence numbers, правило применяется к нему сразу. Весь корпус в память не поднимается. `pandas` строит таблицу на весь набор: на 250k пакетов и 100k сообщений это сотни мегабайт, что упирается в лимит 8 ГБ, а строковая модель плохо выражает провенанс каждого поля к конкретному пакету.
- **Hex-просмотр — собственный, а не готовый виджет.** Требуется не просто показать байты: каждый байт связан с исходным пакетом, подсвечен статусом поля и служит точкой перехода. Готовый hex-виджет такой связи не даёт; свой виджет (`src/ui/hex_view.py`) строит три синхронные области и держит карту «offset → пакет, поле, статус».
- **Wireshark — эталон проверки, а не зависимость.** Собранный поток сравнивается с `Follow TCP Stream` как с внешним арбитром; приложение экспортирует поток в отдельный PCAP (`src/capture/export_wireshark.py`), чтобы это сравнение было возможно без установки Wireshark в рабочий контур. Ни одна функция приложения не требует `tshark` во время работы.

СРЕДА ЗАПУСКА

- **Операционная система:** Linux x86-64.
- **Python:** 3.12 или новее.
- **Графическая среда:** требуется для окна; движок и командные интерфейсы работают без графики.
- **Ресурсы под эталонную нагрузку** (PCAP до 100 МиБ, 250k пакетов, 1000 TCP-сессий, 100k сообщений, ответы до 64 КиБ): 4 vCPU и 8 ГБ ОЗУ. Измеренные значения — в `docs/BENCHMARK.md`.
- **Сеть:** все команды выполняются офлайн.

УСТАНОВКА

Установка пакета и утилит

Выполните (проекту нужен Python 3.12 или новее; в Debian 13 и Ubuntu 24.04 это `python3`):

```
sudo apt-get update
sudo apt-get upgrade
sudo apt-get install -y python3 python3-venv git
git clone https://github.com/ForgeLabHub/ForgeMadrigalProtocolLab
cd madrigal-protocol-lab
```

Установка зависимостей проекта

Зависимости ставятся через виртуальное окружение Python:

```
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -e ".[dev]"
```

Подготовка тестовых данных

Корпус и синтетические фикстуры детерминированы и собираются одной командой:

```
python -m scripts.generate_corpus
```

Команда пишет корпус в `tests/corpus/` и фикстуры в `tests/fixtures/`.

Набор организаторов

Помимо синтетического корпуса в репозитории лежит набор захватов, полученный от организаторов, — `tests/corpus/real/`. Это независимый протокол «термостенд» (`0xa65c`, длина с учётом заголовка): три исследовательских записи, одна дополнительная, восемь диагностических и журнал действий `action_journal.csv`. Разбор и метрики — в `docs/REAL_CORPUS_ANALYSIS.md`, воспроизведение:

```
python -m scripts.investigate_dataset
python -m pytest tests/hypothesis/test_real_corpus.py -q
```

Пример запуска

Командная строка

Нормализация захвата:

```
python -m src.capture.cli --pcap tests/corpus/corpus_capture_01.pcapng \
  --out out/cap01.json --source-name captures/corpus_capture_01.pcapng
```

Полный конвейер (захват → проект → отчёт):

```
python -m src.project.cli pipeline \
  --pcap tests/corpus/corpus_capture_01.pcapng \
  --project project.lab --report reports/REPORT.md
```

Флаг `--verbose` включает журнал чтения, сессий и экспорта в stderr; без него интерфейсы командной строки молчат.

Графический интерфейс

```
python -m src.ui.main_window \
  --capture tests/corpus/reference_export/corpus_capture_01.normalized.json \
  --rule examples/corpus_rule_v1.json
```

Настройки открываются через **Вид → Настройки…** или **Ctrl+,**. При чтении потока **Ctrl +** (или **Ctrl =**) и **Ctrl −** увеличивают и уменьшают байты и дерево сессий, **Ctrl 0** возвращает исходный размер.

Файл открывается двумя разными пунктами меню. **Файл → Открыть нормализованный захват…** читает `*.normalized.json`, а сырой `.pcap`/`.pcapng` нормализует в памяти по ходу открытия. **Файл → Импорт PCAP…** читает сырой захват и дополнительно сохраняет его `*.normalized.json` рядом с исходным файлом, после чего открывает результат.

Веб-интерфейс (необязательно)

Дополнительно есть локальная веб-страница поверх тех же движков захвата и правил. Основным остаётся окно PySide6; веб-слой лишь оборачивает `src.capture` и `src.protocol` и не содержит собственной логики протокола.

```
pip install -e ".[web]"
python -m src.web.app \
  --capture tests/corpus/reference_export/corpus_capture_01.normalized.json \
  --rule examples/corpus_rule_v1.json --port 8765
```

Откройте http://127.0.0.1:8765. Сервер слушает только `127.0.0.1`, всё состояние держит в памяти, базы данных нет. Страница показывает список сессий, шестнадцатеричный вид с подсветкой вердиктов, применение правила, провенанс каждого байта и экспорт самодостаточного HTML-отчёта. Подробности и осознанно исключённый объём — в docs/WEB.md.

Тесты

```
pytest tests/ -q
```

Структура проекта

- `src/capture/` — чтение PCAP/PCAPNG, TCP-сессии, сборка потоков, провенанс
- `src/protocol/` — фрейминг и движок правил
- `src/hypothesis/` — классификация результатов, проверка на корпусе, версии
- `src/ui/` — интерфейс на PySide6
- `src/project/` — переносной проект на диске
- `src/report/` — отчёты об исследовании
- `src/web/` — необязательный локальный веб-интерфейс (FastAPI + HTMX)
- `scripts/` — детерминированные генераторы и бенчмарки
- `docs/` — контракты, схемы, заметки по API, стиль, бенчмарк
- `tests/fixtures/` — небольшие синтетические захваты, по одному поведению
- `tests/fixtures/defects/` — намеренно повреждённые захваты
- `tests/corpus/` — исследовательский корпус и его эталонный экспорт
- `tests/integration/` — прогон корпуса от начала до конца
- `presentation/` — материалы демонстрации и реальные снимки окна

Как пользоваться

Сценарий 1. От захвата к нормализованному JSON

1. Соберите тестовые данные: `python -m scripts.generate_corpus`.
2. Нормализуйте захват командой из раздела «Пример запуска».
3. Сверьте результат с контрактом `docs/CONTRACT.md` и схемами в `docs/schemas/`.

Окно со списком сессий:

Список сессий

Сценарий 2. От гипотезы к правилу и контрпримеру

1. Откройте окно с первым правилом `examples/corpus_rule_v1.json`.
2. Перейдите на вкладку «Интерпретация»: видно `matched` и `mismatched`.
3. Изучите контрпример — конкретные байты, опровергающие узкое правило.

Гипотезы и точность

Точность узкого правила v1: 0.43 (60 support, 80 contradict). Уточнённое правило v2 на том же корпусе: 140 matched, 0 mismatched, точность 1.00. Источник — `docs/METRICS.md`.

Валидация правила

Сценарий 3. От байта к пакету

1. В шестнадцатеричном виде наведите курсор на байт.
2. Всплывающая подсказка показывает смещение, байты и пакет-источник.

Провенанс байта

Сравнение двух версий правила и расхождения между ними:

Сравнение версий

Участники

Сабадаш Павел Предков Никита

- Сабадаш Павел — архитектура, capture, project, report, координация, backend, GUI — @Pasha1778
- Предков Никита — backend, правила, тесты — @petruan

Правила участия и порядок внесения изменений описаны в CONTRIBUTING.md.

Демо

Пошаговый сценарий работы в окне на эталонном корпусе — в docs/demo.md. То же исследование из командной строки, как резервный вариант демонстрации, — в docs/demo_cli.md. Снимки окна лежат в `presentation/screenshots/`, отчёт об исследовании — в REPORT.md.

Документация

- docs/PROJECT_MAP.md — карта проекта: структура, модули, поток данных, решения.
- docs/STACK_COMPLIANCE.md — соответствие технологическому стеку задания: что взято, что заменено и почему.
- ARCHITECTURE.md — подход, модули, форматы, зависимости.
- docs/CONTRACT.md — JSON-контракты между стадиями.
- docs/INSTALL.md — установка из готовых сборок релиза.
- docs/BINARY.md — однофайловые сборки и колесо.
- docs/CAPTURE_API.md — API движка захвата.
- docs/PROTOCOL_API.md — API движка правил и гипотез.
- docs/INTEGRATION.md — стыковка стадий и проверка шва.

- docs/BENCHMARK.md — время и память на эталонной нагрузке.
- docs/METRICS.md — числа с указанием источника.
- docs/DEFENSE_CHECKLIST.md — предполётный список к защите.
- docs/DEMO_CHEATSHEET.md — одностраничник для печати на защиту.
- docs/DEFENSE_FAQ.md и docs/FAQ_FOR_JUDGES.md — вероятные вопросы и ответы.
- docs/BEYOND_SCOPE.md — что реализовано сверх задания и что не является заслугой.
- presentation/Q_and_A.md — пятнадцать вопросов защиты в сжатой форме.
- docs/DEFENSE_FAQ.md и docs/FAQ_FOR_JUDGES.md — вероятные вопросы защиты и ответы.
- docs/SETTINGS.md — диалог настроек и хранимые ключи.
- docs/WEB.md — локальный веб-интерфейс: установка, запуск, объём.
- docs/PORTABILITY.md — проверенные окружения.
- docs/AUDIT.md — трассировка и замечания линтера и типизации.
- docs/SUBMISSION.md — сведения о сдаче.
- docs/ru/README.md — русские версии документов из `docs/`; английский оригинал остаётся источником правды.

Форматы поставки

Основная поставка — исходное дерево с инструкцией выше, упакованное скриптом `python scripts/build_submission.py` в `dist/submission_*.zip`. Архив распаковывается и запускается тем же порядком. Готовые сборки лежат в релизе v1.2.0; пошаговая установка из них — в docs/INSTALL.md.

Формат	Установка	Размер	Команда запуска
AppImage	не требуется	~97 МБ	`chmod +x; ./protocol-lab-*.AppImage`
`.deb`	dpkg	~97 МБ	`sudo dpkg -i madrigal-protocol-lab_*.deb`
`.run`	самозапуск	~97 МБ	`chmod +x; ./madrigal-lab.run`
Binary	portable	~97 МБ	`chmod +x; ./madrigal-lab`
Docker	docker	~451 МБ	`docker load < *.tar.gz`
pip	pip	~15 МБ	`pip install madrigal_protocol_lab-*.whl`
Source	venv	~7 МБ	`pip install -e .`

Бинарник, `.deb`, AppImage, `.run`, образ Docker и колесо — собранные артефакты, в репозиторий не коммитятся; скрипты их сборки коммитятся. Архив проверяется после сборки: он распаковывается во временный каталог, и тесты запускаются уже оттуда.

Ограничения

Область	Состояние
IPv6	Поддержан: `parser.py` разбирает IPv6-датаграммы (включая цепочку extension-заголовков), сессии и потоки строятся как для IPv4, контрольная сумма TCP считается по IPv6-псевдозаголовку.
Фрагментация IP	Собирается для IPv4 и IPv6 (`fragment.py`): датаграмма восстанавливается только при полном покрытии байт и наличии последнего фрагмента. Статусы — `ip_fragment_reassembled`, `ip_fragment_incomplete`, `ip_fragment_overlap`; при конфликтующем перекрытии версия не выбирается молча (IPv4 — первая по времени прихода плюс диагностика, IPv6 — датаграмма отбрасывается по RFC 5722).
Зашифрованные нагрузки	Вне области; байты показываются как есть.
Контрольные суммы	Перестановка трактуется как диагностика.
Эталонные захваты организаторов	Получены и разобраны: `tests/corpus/real/`, результаты — `docs/REAL_CORPUS_ANALYSIS.md`. Синтетический корпус остаётся эталоном для обсуждения фрейминга и воспроизводится из `scripts.generate_corpus`.

Лицензия

MIT, см. LICENSE.

Команды

Команда	Что делает
`make doctor`	python, dpkt/PyYAML, PySide6, libEGL, `QT_QPA_PLATFORM`, ruff, LICENSE
`make install-dev`	`pip install -e ".[dev,web]"`
`make test-core`	446 тестов ядра без Qt (protocol, capture, hypothesis), ~15 с
`make test-web`	веб-интерфейс
`make test-ui`	интерфейс offscreen (нужен PySide6)
`make test`	весь набор, как в CI
`make wheel` / `make package`	сборка wheel и проверка установки в чистом venv
`make docs-check`	полнота документации, живые ссылки, спецификации модулей
`make verify`	lint + ядро + web + документация

Карта всех 53 документов (45 в `docs/` и 8 в корне, на каждом языке) — docs/index.md. Сборка и уровни проверок — BUILD.md. Чем именно доказано каждое утверждение — docs/verification/matrix.md.

Управление проектом: `forge.sh` и `forge.bat`

Один скрипт на Unix-подобные системы (`forge.sh`) и один на Windows (`forge.bat`). Интерфейс одинаковый: те же команды, та же логика, одна реализация на каждой платформе — без bash-скриптов внутри Windows.

```
./forge.sh              # интерактивное меню
./forge.sh doctor       # проверить окружение, ничего не меняя
./forge.sh install      # поставить зависимости
./forge.sh start        # запустить в фоне, дождаться готовности, открыть браузер
./forge.sh status       # что запущено и отвечает ли порт
./forge.sh logs api -f  # логи в реальном времени
./forge.sh test         # тесты
./forge.sh stop         # остановить
```

```
forge.bat
forge.bat start
forge.bat status
```

Что умеет

Команда	Действие
`status`	состояние по портам и HTTP, а не «на глаз»
`start [--fg]`	фон (PID-файлы в `.run/`) или передний план
`stop`	останавливает дерево процессов целиком, порт освобождается
`restart`, `logs [имя] [-f]`, `open`	обычные операции
`doctor`	инструменты, версии, `.env`, `.env` в git, занятость порта
`install`, `test`, `lint`, `build`, `verify`	через `make`, если он есть
`start --web`	веб-интерфейс вместо окна Qt

| `db migrate\|backup\|restore` | база: миграции, копия, восстановление | | `docker up\|down\|logs\|reset` | контейнеры (`down` данные не удаляет) | | `clean [--hard]` | артефакты и логи; `--hard` ещё `.venv` и `node_modules` |

Правила, заложенные в скрипт

- **Ничего не меняет без команды.** `doctor` и `status` только читают.
- **Данные не удаляются.** `stop` и `docker down` сохраняют базу; удаление — только `--hard` или явное подтверждение в `docker reset`.
- **Предполётная проверка.** Если нет `go`/python или занят порт — скрипт говорит об этом сразу, а не через две минуты таймаута.
- **Не висит после падения.** Если процесс умер, запуск прекращается и показывает последние строки лога.
- **Venv главнее системы.** Если есть `.venv`, им пользуются `make`, `pytest` и uvicorn — чтобы `install` и `test` не расходились.

Текущий способ запуска без скрипта: `python -m src.ui.main_window (или --web для браузера)` — адрес –.

Конфигурация: `.env`

Скрипт читает `.env` **собственным парсером**, а не через `source`: файл приходит из репозитория, а выполнять из него произвольный код нельзя. Поддержаны комментарии `#`, пустые строки, префикс `export`, одинарные и двойные кавычки, CRLF, пробелы вокруг `=` и комментарий после значения.

Приоритет значений: **переменная окружения → `.env` → встроенное значение**. Поэтому `PORT=9000 ./forge.sh start` работает независимо от файла.

Команда	Действие
`env` / `env show`	переменные, источник каждого значения и замаскированные секреты
`env init [--force]`	создать `.env` из `.env.example`: заглушки секретов заменяются случайными, права `600`, прежний файл — в `.env.bak`
`env set KEY=VALUE`	записать или заменить одну переменную (атомарно, через временный файл)
`env check`	сверка с `.env.example`: пропавшие переменные, пустые и «примерные» секреты, `.env` в `.gitignore`
`env edit`	открыть файл в `$EDITOR` (fallback: nano → vim → vi)
`env path`, `env reload`	путь к файлу и перечитывание без перезапуска

Секреты в выводе маскируются: `ADMIN_CODE=ch…de (31 симв.)`, пароль внутри строки подключения — `postgres://user:***@host`. Имена вида `*_MINUTES`, `*_URL`, `*_PORT` секретами не считаются: маскировать число минут бессмысленно.

Путь к файлу переопределяется: `FORGE_ENV_FILE=/etc/app.env ./forge.sh status`. В Windows-версии те же команды: `forge.bat env init`, `forge.bat env check`. Ограничение `cmd.exe`: значения, содержащие `%` и `!`, не поддерживаются.

Флаги и автодополнение

```
./forge.sh -h | --help            # справка
./forge.sh -V | --version         # версия скрипта, коммит, состояние .env
./forge.sh -q status              # молчать об успехах
./forge.sh --no-color status      # без ANSI (то же делает NO_COLOR=1)
./forge.sh --json status          # машинный вывод для скриптов и мониторинга

eval "$(./forge.sh completion bash)"   # автодополнение для bash
./forge.sh completion zsh > ~/.zfunc/_forge   # для zsh
```

`test`, `lint`, `build` и `verify` запускаются в окружении **без** переменных из `.env`: иначе локальные проверки вели бы себя иначе, чем CI, и тесты вида «без ключа должен быть fallback» перестали бы что-либо проверять.

Запуск в этом проекте

```
./forge.sh install && ./forge.sh start
```